Hoja de Ejercicios 3.3. Comunicación y sincronización

Ejercicios Prácticos

Los ejercicios marcados con el icono 💻 son prácticos y deben realizarse en el laboratorio. Los ejercicios prácticos de esta hoja requieren el entorno de usuario básico (shell) y de desarrollo (compilador, editores y depurador). Además, algunos ejercicios necesitan acceso de superusuario que está disponible en las máquinas virtuales de la asignatura.

💻 ***Ejercicio 1.*** Escribir un programa que cree un número de threads indicado por el primer argumento, de forma que:

- Cada thread se le asignará un identificador 0,1,2… que imprimirá por la salida estándar y usará para hacer un sleep(3) de los mismos segundos.
- El thread principal esperará a que terminen todos los threads, mostrando el identificador del thread que termina.

💻 ***Ejercicio 2.*** Modificar el programa anterior para que todos los threads esperen el tiempo suficiente para consultar sus identificadores en el sistema. Ejecutar el programa con 4 threads y completar la siguiente tabla con el comando ps, usar las opciones -L (mostrar threads) y la opción de formato -o con los campos adecuados. Identificar el thread principal en la tabla.

PID	TID	TGID	PGID	CMD

💻 ***Ejercicio 3.*** Escribir un programa que realice la suma paralela de los N primeros números naturales. Los argumentos del programa fijarán:

- Primer argumento, el número de threads (N_t).
- Segundo argumento, el tamaño de bloque (T_b) que determina cuántos números sumará cada thread.

Cada thread se le asignará un identificador (i = 0,1,2…) y sumará los enteros en el rango $[T_b \cdot i, (i+1) \cdot T_b - 1]$ que agregará en una variable compartida, suma. El thread principal sincronizará todos los threads y mostrará la suma.

💻 ***Ejercicio 4.*** Escribir un programa con P threads productores (primer argumento) y C threads consumidores (segundo argumento) con las siguientes características:

- El productor escribirá en el buffer el elemento producido y el consumidor simplemente lo imprimirá en el terminal junto con las estadísticas del buffer (in, out y count). Los tiempos de producción y consumición serán de 1 y 2 segundos respectivamente.
- El buffer compartido será un array circular de tamaño fijo que estará representado por una estructura similar a la siguiente:

```
typedef struct _buffer
{
    int count;
    int in;
    int out;
    int data[BUFFER_SIZE];
} buffer_t;
```

- Implementar el sistema con dos variables de condición que estarán asociadas a los siguientes predicados:
  - Thread consumidor: *"puedo consumir"*, count > 0

    - Thread producir: *"puedo producir"*, count < BUFFER_SIZE
- Cada productor producirá un número fijo de elementos (NUM_ELEMENTS).

💻 ***Ejercicio 5.*** Añadir una condición de finalización al ejercicio 4 usando el patrón centinela o *píldora envenenada*, de la siguiente forma:

- El thread principal sincronizará la ejecución de todos los productores para asegurarse de que todos los elementos se han producido.
- A continuación escribirá los elementos (píldoras envenenadas, por ejemplo valor -1) en el buffer, tantos como consumidores se hayan creado. Nota: Debe preservar el acceso concurrente al búfer, igual que el thread productor.
- Cuando un consumidor lea la píldora envenenada del búfer terminará después de ajustar los índices y señalizar la variable de condición.

💻 ***Ejercicio 6.*** Escribir un programa que cree L lectores (primer argumento) y E escritores (segundo argumento), de forma que:

- Los threads de tipo *Lector* imprimirán por pantalla un entero compartido y esperarán 0.1s con la llamada usleep(3). Este acceso lo repetirán 5 veces.
- Los threads de tipo *Escritor* incrementarán en 1 la variable y esperarán 0.25s. Este acceso lo repetirán 3 veces.
- El thread principal arrancará primero los thread de tipo *Escritor* y sincronizará la finalización de todos los threads lectores y escritores.

Ejercicio 7. Diseña una implementación de los mutex de lectura/escritura de la biblioteca Posix Threads (pthread_rwlock_[rd|wr][lock|unlock]) usando mutex normales y variables de condición. La implementación debe dar preferencia a los escritores.

💻 ***Ejercicio 8.*** Utilizando como base el programa del ejercicio 4, diseñar e implementar una aplicación concurrente en C que explore un directorio del sistema de ficheros y compute el tamaño total de los ficheros regulares encontrados.

La aplicación aceptará dos argumentos, el primero será el directorio del que se quiere explorar; y el segundo será el número de threads Consumidores. Los threads de la aplicación implementarán la siguiente lógica:

Thread productor

- Abre el directorio indicado y recorre sus entradas usando opendir() y readdir().
- Inserta las rutas completas de los ficheros o subdirectorios en un búfer circular compartido (usar snprintf())

Threads consumidores

- Extraen rutas del búfer. Además, la llamada al sistema para calcular el tamaño no debe mantener bloqueado el búfer compartido, así que el thread copiará la ruta en una variable local con strcpy().
- Una vez realizada la copia, actualizados los índices del búfer y señalizada la variable de condición usará lstat() para obtener información del fichero. Si es un fichero regular acumulará su tamaño en una variable global protegida por otro mutex.
- Si el elemento es una "píldora venenosa" (cadena vacía) finalizará su ejecución.

Thread principal

- Creará el thread productor y los threads consumidores.
- Cuando el productor termina de recorrer el directorio (pthread_join), inserta una píldora venenosa por cada consumidor para indicar el fin del trabajo.
- Finalmente espera (pthread_join) la terminación de todos los consumidores y muestra el tamaño total acumulado de los ficheros procesados.

Comprobar el funcionamiento del programa con diferentes niveles de concurrencia y la orden time(1) para evaluar el tiempo de ejecución en cada caso. Se pueden utilizar estructuras similares a las siguientes para gestionar los datos del programa:

```
#define BUFFER_SIZE 10

typedef struct _buffer_t
{
```

```
    pthread_mutex_t mutex;
    pthread_cond_t produce;
    pthread_cond_t consume;
    int count;
    int in;
    int out;
    char data[BUFFER_SIZE][PATH_MAX];
} buffer_t;

typedef struct _lint_mutex
{
    pthread_mutex_t mutex;
    long total_size;
} lint_mutex_t;
```

Ejercicio 9. Considera el siguiente programa, que pretende resolver el problema de la sección crítica para dos threads mediante un esquema puramente software que no recurre a ningún servicio de sincronización del sistema operativo:

```
void thread1()
{
    while(TRUE){
        // Espera activa
        while(turno != 0);
        <<Sección crítica T1>>
        turno = 1;
        <<Más código T1>>
    }
}

void thread2()
{
    while(TRUE){
        // Espera activa
        while(turno != 1);
        <<Sección crítica T2>>
        turno = 0;
        <<Más código T2>>
    }
}
```

¿Resuelve correctamente el programa el problema de la sección crítica, garantizando exclusión mutua, progreso y espera limitada (ausencia de inanición)? Justifica la respuesta.

💻 ***Ejercicio 10.*** Escribe un programa concurrente formado por tres threads que se comuniquen entre sí. El thread 1 generará los primeros 1000 números pares y el thread 2 se encargará de generar los primeros 1000 números impares. El thread 3, por su parte, leerá (consumirá) estos números y los imprimirá por pantalla. El programa concurrente debe garantizar que los números mostrados por pantalla aparecen en orden: 1,2,3,4,5… Impón la sincronización necesaria usando:

a) Cerrojos y variables de condición
b) Semáforos

Ejercicio 11. Implementa un semáforo general usando mutex, variables de condición y otras variables compartidas (por ejemplo, enteros). Para ello, debe definirse el nuevo tipo de dato en C mediante una definición de estructura y su correspondiente typedef. Las operaciones wait y signal sobre el semáforo deben implementarse como dos funciones independientes que acepten un puntero al tipo del semáforo como argumento.

Ejercicio 12. Modifica la implementación del Ejercicio 7 para dar preferencia a los lectores en lugar de a los escritores: ningún lector debe quedar esperando si hay otros lectores dentro de la sección crítica (con esta política, un escritor puede llegar a sufrir inanición si los lectores no dejan de llegar).

Ejercicio 13. Da una solución al problema de los filósofos comensales usando un monitor (un mutex, variables de condición y otras variables compartidas). El monitor debe implementar dos procedimientos, getChopsticks(i) y putChopsticks(i), que permitan a un filósofo (identificado por el

parámetro i) coger los dos palillos asociados y devolverlos a la mesa, respectivamente. El siguiente fragmento de código representa el comportamiento de un filósofo (thread):

```
void philosopher(int i) {
    while(1){
        think();
        /* Pide permiso al monitor para coger los palillos */
        getChopsticks(i);
        eat();
        /* Indica al monitor que devuelve los palillos a la mesa */
        putChopsticks(i);
    }
}
```

Ejercicio 14. El problema de los fumadores de cigarrillos fue presentado originalmente por Suhas Patil. En este problema intervienen cuatro threads: un agente y tres fumadores. Los fumadores repiten indefinidamente el ciclo de esperar los ingredientes y después liar y fumar un cigarrillo. Para que un fumador pueda liar y fumar un cigarrillo necesita tres ingredientes: tabaco, papel y cerillas. Se asume que el agente dispone de una cantidad infinita de los tres ingredientes, y que cada fumador dispone de una cantidad infinita de uno de ellos: un fumador tiene cerillas, otro tiene papel y el tercero tiene tabaco.

El agente elige repetidamente dos ingredientes distintos al azar y los pone a disposición de los fumadores dejándolos sobre la mesa. Según qué ingredientes se hayan elegido, el fumador que posee el ingrediente complementario recoge los otros dos y procede a fumar.

Escribe un programa concurrente que sincronice al agente y a los fumadores mediante un monitor. Se asume que el agente no tiene forma alguna de saber qué ingrediente tiene cada fumador.

Ejercicio 15. El problema de los salvajes comensales. Una tribu de salvajes come de una gran olla comunitaria de guiso que puede contener M raciones. Cuando un salvaje quiere comer, se sirve de la olla, salvo que esté vacía. Si la olla está vacía, el salvaje despierta al cocinero y espera a que este la rellene.

Varios threads salvaje ejecutan el siguiente código:

```
while (True){
    getServingFromPot()
    eat()
}
```

El thread cocinero se comporta así:

```
while (True){
    putServingsInPot(M)
}
```

Las restricciones de sincronización son las siguientes:

- Los salvajes no pueden invocar getServingFromPot() si la olla está vacía.
- El cocinero solo puede invocar putServingsInPot() si la olla está vacía.

Añade el código necesario para los salvajes y el cocinero de forma que se cumplan las restricciones de sincronización, usando:

a) Mutex, variables de condición y otras variables compartidas, como enteros o booleanos.
b) Semáforos y otras variables compartidas, como enteros o booleanos.

Ejercicio 16. Crea un programa concurrente que emule el comportamiento de una gasolinera con dos surtidores que permite a los clientes pagar con tarjeta. Cuando un cliente usa un surtidor, se sirve de él invocando una función *void PumpFuel(int pump_id, int price)*. Cuando termina, el surtidor queda libre para que lo use otro cliente. Para que la gasolinera sea justa con los clientes deben cumplirse las siguientes restricciones:

- Los surtidores deben asignarse a los clientes por orden de llegada.
- Dos clientes no pueden usar el mismo surtidor simultáneamente.
- Ningún cliente debe esperar innecesariamente mientras haya un surtidor libre.

Cada cliente, modelado como un thread del programa concurrente, se comporta así:

```
void customer(int price) {
    int pump;
    pump = getUnusedPump();
```

```
    /* Usando el surtidor */
    PumpFuel(pump, price);
    /* Termina de usar el surtidor */
    releasePump(pump);
}
```

Usando mutex, variables de condición y otras variables compartidas, implementa las funciones getUnusedPump() y releasePump() para imponer las restricciones de sincronización anteriores. Ten en cuenta que la función getUnusedPump() debe bloquear a un cliente hasta que se le asigne un surtidor libre.

Ejercicio 17. Se quiere simular el trabajo en el muelle de carga de un almacén, usando threads para simular a los operarios que cargan paquetes y a los camiones que se cargan. El siguiente código describe la estructura de los threads usados para operarios y camiones:

```
#define NP 10 // Capacidad máxima de camión

void *truck(void *arg)
{
    while (1) {
        // esperar a poder entrar
        enter_dock();
        // esperar a camión cargado
        delivery();
    }
}

void *worker(void *arg)
{
    while (1) {
        p = get_parcel();
        // esperar a poder cargar
        load_parcel(p);
        // notificar si cargado
    }
}
```

Los operarios repiten indefinidamente las acciones de coger un paquete (get_parcel) y cargarlo en el camión (load_parcel). Varios operarios pueden estar cogiendo un paquete a la vez, pero deben cargar el camión de uno en uno. El thread operario solo debe ejecutar la función load_parcel cuando haya un camión en el muelle y quede espacio en dicho camión para el paquete.

Los camiones tienen capacidad para NP paquetes, y repiten indefinidamente las acciones de entrar al muelle (enter_dock) para recoger la carga y después salir a reparto (delivery). Varios threads camión pueden estar ejecutando la función delivery a la vez. Sin embargo, solo hay sitio para un camión en el muelle, así que el thread que simula al camión debe esperar a que el muelle quede libre para ejecutar la función enter_dock. Por otro lado, el thread debe esperar a que el camión esté completamente cargado para salir del muelle e iniciar el reparto.

Completa las funciones truck y worker para que los threads se sincronicen correctamente, añadiendo las variables que consideres necesarias.

Ejercicio 18. Se quiere simular un pequeño torneo de póker entre amigos usando threads POSIX. La función de entrada de los threads que simulan a los jugadores tiene la siguiente estructura:

```
void play(void);

void *player(void *arg)
{
    while (1) {
        seat_at_table();
        play();
        leave_table();
    }
}
```

La función seat_at_table debe garantizar que los jugadores acceden a la mesa en estricto orden de llegada, mientras la ronda no haya empezado. Además, los threads no deben ejecutar la función play hasta que los 4 jugadores que disputarán la ronda estén sentados a la mesa (hayan invocado

seat_at_table). Por otro lado, los threads de los 4 jugadores que juegan juntos deben poder ejecutar la función play de forma concurrente.
Cuando termina la ronda (retorno de la función play) el último jugador en abandonar la mesa (leave_table) debe indicar al resto que la ronda actual ha terminado y que puede empezar una nueva. Implementa las funciones seat_at_table y leave_table para que los threads se sincronicen correctamente, añadiendo las variables globales que necesites.
💻 ***Ejercicio 19.*** Escribe dos programas en C, un productor y un consumidor, que se ejecuten como dos procesos independientes (no como threads de un mismo proceso) y se comuniquen mediante una región de memoria compartida POSIX, siguiendo el patrón visto en teoría:

- El productor debe: crear un fichero con open() y reservarle espacio con ftruncate(), proyectarlo en su espacio de direcciones con mmap() (flags MAP_SHARED), crear dos semáforos con nombre con sem_open() (uno para huecos libres y otro para elementos disponibles), y usar la región de memoria compartida como buffer circular para producir una serie de elementos.
- El consumidor debe: esperar a que el fichero exista y abrirlo, proyectarlo también con mmap(), abrir (no crear) los mismos semáforos con nombre, y consumir los elementos producidos en el mismo orden en que se generaron.
- Al terminar, el proceso consumidor debe liberar los recursos con munmap(), shm_unlink() y sem_unlink().

Comprueba que ambos procesos (verificables por tener PID distinto) se comunican correctamente y que no se pierde ni se duplica ningún elemento.